SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 1 - CO1

CO1 Coverage: *Operational vs decision-support systems, data warehouse architecture, ETL, dimensional data modelling (star and snowflake schemas), data cubes, OLAP operations, query processing, open-source BI tools and KNIME for reporting.*

Course Outcome: CO1 – Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Assignment Title: Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain (Total Marks - 50)

Duration: 1 week

Scenario — FreshCart Retail Analytics

FreshCart operates 120 supermarkets across six states. Every sale, return, loyalty-card swipe and promotion is recorded in a normalised operational (OLTP) database containing the tables Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns.

Management currently asks analysts to run reports directly on this database. Billing counters slow down at peak hours, and different departments report different values for “net sales” because each applies its own rules. Management has asked you to design a **data warehouse** that supports questions such as:

- Which product categories show declining net sales, quarter over quarter, in each region?
- Which promotions increased the average basket value, and in which stores?
- How does weekend sales performance differ from weekday performance for each department?
- Which customer segments return the most items, and of which products?
- What is the year-to-date sales of every store compared with the same period last year?

The sales fact data may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ sales line items

You are required to investigate, design and compare alternative warehouse designs, and to demonstrate OLAP operations on the design you recommend. Your solution must remain practical as data grows. Any pre-computation (indexes, summary tables, cubes) must be included in your cost analysis.

Design Challenge:

FreshCart now launches an online channel that sends about 20,000 orders per day from a new source system. Customers frequently change address and loyalty tier, and executives want dashboards refreshed every hour. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Analyse the Decision-Support Requirement **4 Marks**

Examine why FreshCart’s existing operational system is unsuitable for management analysis, and define what the warehouse must support.

Discuss:

- At least **eight business questions** (including the five above) classified as strategic or tactical
- For each question, the **measures** and **dimensions** it requires
- How the operational system and a decision-support system differ in purpose, data currency, structure, query pattern, users and volume
- Which of your questions cannot be answered efficiently from the OLTP database, and why

Do not simply provide a table of textbook differences. You are expected to **relate each difference to FreshCart's situation**.

Evidence Expected
Business questions with their measures and dimensions, and a justified explanation of why a separate decision-support system is required for FreshCart.

TASK 2 – Propose the Warehouse Architecture **3 Marks**

Propose an architecture for FreshCart and represent it using a labelled diagram.

Your architecture must show:

- Data sources, staging area, ETL process, warehouse storage and metadata repository
- Data marts, if any, and the OLAP server
- Front-end tools for reporting, query and analysis
- Your choice among an enterprise warehouse, independent data marts, dependent data marts or a virtual warehouse, and **why**

A copied textbook architecture without justification for FreshCart's needs will **not** receive full credit.

Evidence Expected
A labelled architecture diagram together with a reasoned justification of the selected architecture type.

TASK 3 – Design the Star Schema **6 Marks**

Design a dimensional model for FreshCart's sales analysis using a **star schema**.

You must:

- State the **business process** and declare the **grain** of the fact table *before* listing measures
- Identify the fact table and its measures, and classify each as **additive, semi-additive or non-additive**
- Identify the dimension tables (at minimum Date, Product, Store, Customer and Promotion) with their attributes and **concept hierarchies**
- Define primary keys, surrogate keys and foreign keys
- Show how a second fact table (for example Returns) can share **conformed dimensions** with the Sales fact table
- Provide the schema diagram and the table-creation (DDL) statements

You must **declare the grain explicitly** and show that every measure and dimension is consistent with it.

Evidence Expected
A star-schema diagram, declared grain, classified measures, dimension hierarchies, keys and DDL statements consistent with the declared grain.

TASK 4 – Compare with a Snowflake Schema **4 Marks**

Create a snowflaked version of the Product dimension (product → brand → category → department) and the Store dimension (store → city → state → region).

Assume 50,000 products, 800 brands, 60 categories, 12 departments, 120 stores, 70 cities, 6 states and 3 regions.

Discuss:

- The additional tables, keys and joins introduced
- The estimated change in dimension storage (show your calculation)
- The number of joins needed for three representative queries in the star and snowflake versions

- Effect on query performance, ETL maintenance and ease of understanding for business users
- Which design you would choose for FreshCart, and under what condition you would change that decision

You are expected to **support the comparison with calculations and examples**, rather than repeat that "snowflake saves space but is slower".

Evidence Expected
A snowflake design, storage and join-count calculations, and a justified schema choice.

TASK 5 – Design the ETL Process **5 Marks**

The following extract from the source systems is to be loaded into the warehouse:

Order ID	Customer	Store	Product	Qty	Unit Price	Order Date
1001	Rajesh Kumar	Chennai-04	Basmati Rice 5kg	2	620.00	03/11/2026
1002	R. Kumar	chennai-04	Basmati rice 5 kg	1	620.00	2026-11-03
1003	Anita Joseph	Pune-11	Olive Oil 1L	-3	780.00	04-Nov-26
1004	Meena S	(null)	Tea 250g	4	95.50	04/11/2026
1005	Anita Joseph	Pune-11	Olive Oil 1L	3	9.40 (USD)	04/11/2026
1006	Vikram Rao	Hyd-02	(null)	1	1,250	05/11/2026

Design the ETL process for loading this data.

Discuss:

- Every data-quality problem in the extract and the **transformation rule** that resolves it
- Surrogate-key lookup and how unknown or late-arriving dimension members are handled
- How a change of customer address will be handled using a **slowly changing dimension** technique (Type 1, 2 or 3) and why
- The difference between the **initial load** and the **incremental load**, and the order in which dimensions and facts are loaded
- Handling of rejected records and audit of row counts

Show the **before and after** version of the extract. Merely listing ETL stage names will not receive full credit.

Evidence Expected
An ETL flow diagram, transformation rules for each defect, SCD decision with justification and the cleaned version of the sample extract.

TASK 6 – Model the Data Cube and Apply OLAP Operations **5 Marks**

FreshCart's analysts view sales as a data cube with these dimensions and hierarchy levels:

- **Time:** day → month → quarter → year
- **Product:** product → category → department
- **Store:** store → city → state
- **Customer segment:** segment (no hierarchy)

Discuss:

- The total number of cuboids in the lattice **with** the hierarchies above, and **without** them (show the calculation)
- The base cuboid and the apex cuboid, and what each contains
- One business question for each of **roll-up, drill-down, slice, dice and pivot**, with the cuboid it uses and an equivalent SQL statement
- Which cuboids you would **materialise** if storage allows only a small fraction of the lattice, and the reasoning for your selection

You are expected to **connect each OLAP operation to a specific FreshCart business question**, rather than define the operations generally.

Evidence Expected
Cuboid calculations, five OLAP operations with queries, and a justified partial-materialisation choice.

TASK 7 – Develop an OLAP Application **5 Marks**

Develop an application that performs OLAP operations on your warehouse. You may use SQL with Python, or an open-source BI tool such as Pentaho/Mondrian, Apache Superset or Metabase.

Ensure that:

- The warehouse is populated with realistic (real or synthetic) data of about **1,000 → 10,000 → 100,000 → 1,000,000 fact rows**, and the data-generation method is documented
- The user can choose the operation, dimension, level and filter values **through input**, rather than running fixed queries
- Roll-up, drill-down, slice, dice and pivot are all demonstrated
- Results are **cross-checked** against an independent calculation (for example, grand totals reconcile with the source data)

The purpose of the application is **not merely to display charts**, but to demonstrate that your dimensional model supports the intended analysis.

Evidence Expected
Working application with screenshots, source code, data-generation script and a reconciliation of results.

TASK 8 – Evaluate Query Processing Performance **4 Marks**

Select three representative analytical queries. Execute each on:

- (i) the normalised OLTP-style schema
- (ii) your star schema
- (iii) your star schema with indexes and/or a summary (aggregate) table

Record, for each data size:

- Execution time
- Rows scanned or rows read
- Number of joins and the query plan
- Storage and load time of the additional structures

Explain why the observed differences occur and what trade-off the summary structures introduce. Execution time must not be the only evidence presented.

Evidence Expected
Tables/graphs of results, query plans and an interpreted comparison.

TASK 9 – Build a KNIME Reporting Workflow **3 Marks**

Build a KNIME workflow that reads warehouse data (CSV or database), cleans it, aggregates it and produces a management report.

Discuss:

- The nodes used (for example File/DB Reader, Joiner, GroupBy, Pivoting, Row Filter, charts) and the purpose of each
- How each part of the workflow corresponds to ETL or OLAP concepts from Tasks 5 and 6
- One limitation of the tool when the data grows to 1,000,000+ rows

Evidence Expected
Annotated workflow screenshot, the exported report and a short explanation of node-to-concept mapping.

TASK 10 – Recommend and Defend the Final Design **3 Marks**

Recommend the final warehouse design for FreshCart.

Discuss:

- The schema and architecture you recommend, and why they are better than the alternatives you studied
- The evidence from Tasks 4 and 8 supporting your choice
- Trade-offs, load cost and limitations
- Whether the design remains practical at 1,000,000+ rows and beyond

Your recommendation must be based on **evidence obtained from your own design and experiments**.

Evidence Expected
A technically justified final design recommendation supported by your analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 11 – Re-evaluate and Adapt the Design **8 Marks**

Changed Requirement
FreshCart launches an online channel. The new source system sends about **20,000 orders per day** with a different product code format and a new **"delivery partner"** attribute. Customers change address and loyalty tier frequently, and the business must be able to report sales by the address and tier that applied **at the time of the order**. Executives want dashboards **refreshed every hour**.

Re-evaluate your original design in response to these changed conditions.

You may:

- Retain the design
- Modify the design
- Redesign parts of the warehouse or ETL process

Whatever you decide, you must **justify it with evidence**.

A. Impact Analysis – 2 Marks
Identify which parts of your schema, ETL, cubes and architecture are affected by the changes, and which are not. Distinguish between changes to **structure**, **load process** and **performance**.

B. Evaluation of the Original Design – 2 Marks
Test your original design against the new requirement, for example by simulating the new order volume and the hourly refresh. Identify where the original design is adequate and where it fails.

C. Adaptation or Redesign – 2 Marks
Present the revised design: for example, a new conformed dimension or fact table, SCD handling, incremental or micro-batch ETL, or additional aggregates. Show the revised schema or ETL flow.

D. Evidence-Based Defence – 2 Marks
Defend your decision using measurements or calculations (load time, query time, storage) from your own experiments. State what you would monitor after deployment.

Evidence Expected
A reasoned impact analysis, an evaluation of the original design, a revised design and an evidence-based defence of the final decision.

MARK DISTRIBUTION

Task	Marks
Task 1 – Decision-Support Requirement	4
Task 2 – Warehouse Architecture	3
Task 3 – Star Schema Design	6
Task 4 – Snowflake Comparison	4
Task 5 – ETL Design	5

Task	Marks
Task 6 – Data Cube and OLAP Operations	5
Task 7 – OLAP Application	5
Task 8 – Query Processing Performance	4
Task 9 – KNIME Reporting Workflow	3
Task 10 – Final Design Recommendation	3
Subtotal	**42**
Task 11 – Day-4 Design Challenge	**8**
TOTAL	**50**

EXPANDED SUBMISSION VERSION

This revision expands the explanations, design reasoning, validation approach and implementation guidance under each task. Replace student-detail placeholders and attach your own application/KNIME screenshots and measured benchmark evidence before submission.

EXECUTIVE SUMMARY

This assignment proposes a practical analytical warehouse for FreshCart, a retail chain with 120 supermarkets in six states. The current OLTP database is designed for fast transaction processing, not repeated management analysis. The recommended solution is an enterprise data warehouse with a dimensional star schema, conformed dimensions, controlled ETL, a separate returns fact table, and a lightweight OLAP reporting application. A snowflake alternative is evaluated quantitatively. The design also addresses the new online channel, hourly dashboard refresh, customer history and delivery-partner analysis.

Important implementation note: the supplied assignment brief requires actual application screenshots, measured query timings, query plans and a KNIME workflow. This document supplies the design, runnable starter implementation and exact validation procedure. Timings and screenshots must be captured by running the scripts in the student's environment; no experimental measurements are fabricated.

TASK 1 – ANALYSE THE DECISION-SUPPORT REQUIREMENT (4 MARKS)

1.1 Why the current OLTP system is unsuitable

FreshCart's operational database records individual orders, order lines, customers, products, stores, employees, promotions, suppliers and returns. Its main purpose is to complete billing and update transactions accurately. Management reports repeatedly join these normalized tables and scan detailed records. During peak hours, analytical queries compete with checkout transactions for CPU, memory, disk I/O and locks. This explains the observed billing slowdown.

- Different departments calculate net sales differently. A warehouse provides one governed definition: Net Sales = gross line sales − discounts − refunds/returns, with tax treatment and currency conversion explicitly documented.
- Operational tables usually retain current customer attributes. If a customer's address or loyalty tier changes, historical reports may incorrectly show the current value for old purchases unless history is preserved.
- The normalized structure requires many joins for questions such as sales by category, region, quarter and promotion. These joins are expensive over millions of order lines.
- OLTP queries are short, selective and write-heavy; analytical queries are read-heavy, aggregate large time ranges and compare multiple dimensions.

1.2 Business questions, classification, measures and dimensions

Business question	Type	Measures	Required dimensions
1. Which categories have declining net sales quarter over quarter in each region?	Strategic	Net sales; QoQ change %	Time (quarter/year), Product (category/department), Store (region)
2. Which promotions increased average basket value, and in which stores?	Tactical	Basket value; average order value; uplift %	Promotion, Store, Date, Customer segment
3. How does weekend sales performance differ from weekdays by department?	Tactical	Net sales; quantity; order count	Date (day type/week), Product (department), Store
4. Which customer	Strategic	Returned quantity; return	Customer segment,

segments return the most items and which products are returned?		value; return rate	Product, Store, Date, Return reason
5. What is each store's YTD sales compared with the same period last year?	Strategic	YTD net sales; prior-year YTD; variance %	Date, Store, Region
6. Which stores have the highest average transaction value this month?	Tactical	Net sales; distinct order count; average basket value	Date (month), Store, Region
7. Which products have low sales but high return rates?	Tactical	Units sold; units returned; return rate	Product, Category, Store, Date
8. Which loyalty tiers contribute the greatest revenue over the year?	Strategic	Net sales; order count; customer count	Customer, Loyalty tier, Date, Store
9. What is supplier/brand sales contribution by department?	Strategic	Net sales; quantity sold	Product, Brand, Department, Date
10. How do online and in-store channels compare by delivery partner and region?	Strategic	Net sales; order count; delivery time (if captured)	Sales channel, Delivery partner, Store/fulfilment location, Date

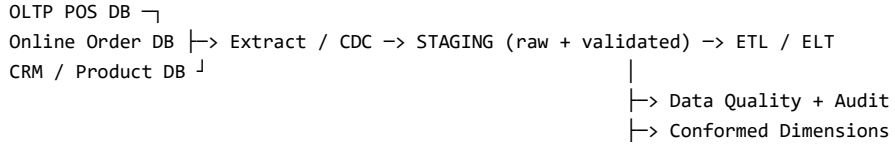
1.3 OLTP versus decision-support system in FreshCart

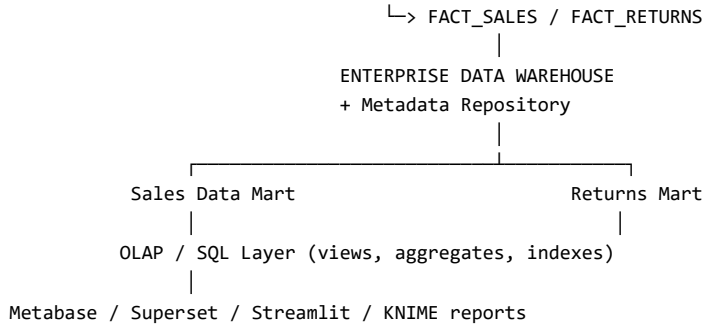
Aspect	FreshCart OLTP	FreshCart decision-support warehouse
Purpose	Capture orders, payments, returns and stock updates reliably.	Explain trends, compare periods, evaluate promotions and support decisions.
Currency	Current operational state; transaction updates happen continuously.	Historical, integrated snapshots; refreshed on a defined schedule (hourly target).
Structure	Normalized tables to reduce duplication and protect transaction integrity.	Dimensional facts and dimensions optimized for aggregation and understandable analysis.
Query pattern	Short inserts/updates and point lookups at checkout.	Large scans, joins, grouping, filtering and time comparisons.
Users	Cashiers, store operators, order-processing services.	Executives, category managers, regional managers and analysts.
Workload/volume	Many concurrent small transactions.	Fewer but resource-intensive queries over millions of line items.

Questions 1, 3, 4, 5, 7, 8 and 10 are particularly inefficient on OLTP because they require historical comparisons, multiple joins, grouping across large populations, and consistent historical customer attributes. A separate warehouse isolates analytical workload, stores history, standardizes business definitions and allows indexes/aggregates to be tuned without slowing billing.

TASK 2 – PROPOSE THE WAREHOUSE ARCHITECTURE (3 MARKS)

2.1 Recommended architecture





2.2 Architecture choice and justification

Choose an enterprise data warehouse with dependent data marts. All six-state stores and the new online channel load into one governed warehouse. Sales and returns marts are derived from conformed dimensions rather than built independently. This prevents each department from creating a different net-sales definition and allows cross-channel reporting. A virtual warehouse would continue to query operational sources and would not sufficiently isolate peak-hour billing. Independent marts would duplicate customer/product definitions and create reconciliation problems.

The staging area preserves source extracts for replay and auditing. ETL performs standardization, validation, currency conversion, surrogate-key assignment and slowly changing dimension handling. The metadata repository stores source-to-target mappings, definitions, refresh timestamps, lineage and data-quality rules. Hourly incremental loads are scheduled after the online channel is introduced.

2.3 Data flow in detail

Source extraction: POS and store databases provide completed sales, order lines, returns and promotion usage. The online order platform contributes e-commerce orders. CRM provides customer and loyalty attributes, while product master data provides product codes and category mappings. Extraction should use change-data-capture (CDC) or a reliable modified-timestamp watermark so that only new and changed records are moved after the initial historical load.

Staging: raw source records are retained without overwriting them. Separate raw, cleansed and ready-to-load areas make it possible to trace a warehouse value back to the original record. Every batch receives a batch identifier, source name, extraction timestamp and file or table reference. This is important when departments question a sales figure or when a failed batch must be replayed.

Transformation and integration: the ETL layer standardizes date formats, store codes, product identifiers, currencies and customer identities. It validates required fields, applies business rules for discounts and returns, resolves dimension surrogate keys, and calculates approved measures. Rejected records are stored with a reason code instead of being silently discarded.

Warehouse and presentation: dimensions are loaded before facts so that fact foreign keys can be resolved. Sales and returns are stored at their own declared grains. The OLAP layer exposes consistent views and selected aggregate tables. Metabase or Superset can provide dashboards, while KNIME can support repeatable data preparation and scheduled reporting.

2.4 Metadata, governance and refresh policy

The metadata repository should document the meaning of each field, source-to-target mapping, data owner, refresh frequency, transformation rule, surrogate-key policy, SCD policy and lineage. For example, the definition of net sales must state whether tax is included, how discounts are allocated to lines, how refunds are treated and which exchange-rate date is used.

FreshCart should publish a dashboard only after a batch passes row-count and amount reconciliation. An hourly schedule can be implemented as an hourly micro-batch: extract changes, stage them, validate, update dimensions, load facts, refresh selected aggregates and then expose the new batch. If a batch fails validation, the previous successful dashboard remains available and the failed batch is flagged.

TASK 3 – DESIGN THE STAR SCHEMA (6 MARKS)

3.1 Business process and grain

Business process: retail sales analysis. Grain: one row in FACT_SALES represents one product line on one completed sales order at one store/channel, for one customer, on one transaction date, under one promotion (or the 'No Promotion' member). A line with quantity 2 remains one fact row with quantity 2; it is not split into two rows.

3.2 Fact table and measures

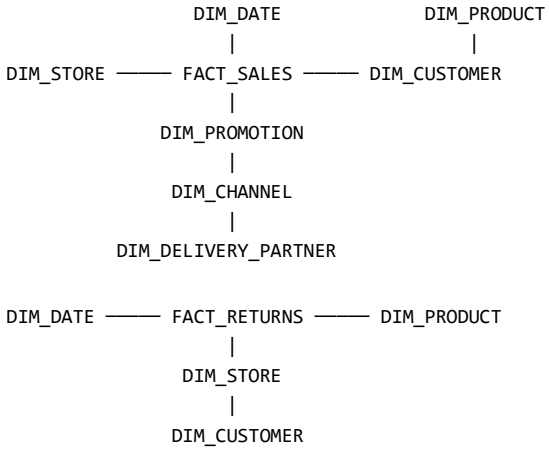
Measure	Meaning	Additivity
quantity_sold	Units sold on the line.	Additive across product, store, customer and time.
gross_sales_amount	quantity × unit selling price before discounts and returns.	Additive.
discount_amount	Discount allocated to this order line.	Additive.
net_sales_amount	Gross sales − discount − allocated refund/return amount, according to the agreed policy.	Additive; use one common definition.
unit_selling_price	Price per unit on the transaction.	Non-additive; do not sum. Use weighted average where appropriate.
discount_rate	discount_amount / gross_sales_amount.	Non-additive; recompute from summed components.
order_count	Distinct orders; do not sum line-level 1s across arbitrary joins without distinct order logic.	Non-additive/semi-additive depending on stored design.

For reliable order counts, use COUNT(DISTINCT order_number) in queries or create a separate order-header fact if order-level measures are required. Net sales should be computed consistently from gross sales, discounts and returns. The sample DDL stores line-level net sales before later returns; a reporting view subtracts the conformed returns fact to prevent double counting.

3.3 Dimensions, attributes and hierarchies

Dimension	Key and example attributes	Concept hierarchy
DIM_DATE	date_key (PK), full_date, day, day_name, weekend_flag, month, month_name, quarter, year, fiscal_period	Day → Month → Quarter → Year
DIM_PRODUCT	product_key (PK), source_product_code, product_name, brand_name, category_name, department_name, package_size, active_flag	Product → Category → Department
DIM_STORE	store_key (PK), store_code, store_name, city, state, region, store_type	Store → City → State → Region
DIM_CUSTOMER	customer_key (PK), customer_id, customer_name, segment, loyalty_tier, address, city, state, effective dates, current_flag	Customer → Segment; customer history via SCD Type 2
DIM_PROMOTION	promotion_key (PK), promotion_code, promotion_name, promotion_type, start_date, end_date	Promotion → Promotion type
DIM_CHANNEL	channel_key (PK), channel_name (Store/Online), source_system	Channel (no hierarchy)
DIM_DELIVERY_PARTNER	delivery_partner_key (PK), partner_code, partner_name	Partner (no hierarchy; primarily online)

3.4 Star schema diagram



FACT_RETURNS uses the same date, product, store and customer dimensions (conformed dimensions). It also has return_reason_key if return reasons are available. Its grain is one returned product line per return transaction. This separate fact avoids mixing sales-event and return-event grains.

3.5 Keys and DDL

Each dimension uses an integer surrogate primary key generated by the warehouse. Source business keys are retained as alternate identifiers for ETL lookup. Fact foreign keys reference surrogate keys. The date key may be an integer YYYYMMDD, while other dimension keys are generated integers.

```
CREATE TABLE dim_date (
 date_key INT PRIMARY KEY, full_date DATE NOT NULL, day_num TINYINT,
 day_name VARCHAR(12), weekend_flag BOOLEAN, month_num TINYINT,
 month_name VARCHAR(12), quarter_num TINYINT, year_num SMALLINT
);
CREATE TABLE dim_product (
 product_key INT AUTO_INCREMENT PRIMARY KEY, source_product_code VARCHAR(40) NOT NULL,
 product_name VARCHAR(120), brand_name VARCHAR(80), category_name VARCHAR(80),
 department_name VARCHAR(80), package_size VARCHAR(40), active_flag BOOLEAN,
 UNIQUE(source_product_code)
);
CREATE TABLE dim_store (
 store_key INT AUTO_INCREMENT PRIMARY KEY, store_code VARCHAR(30) NOT NULL UNIQUE,
 store_name VARCHAR(100), city VARCHAR(60), state_name VARCHAR(60),
 region VARCHAR(40), store_type VARCHAR(30)
);
CREATE TABLE dim_customer (
 customer_key INT AUTO_INCREMENT PRIMARY KEY, customer_id VARCHAR(40) NOT NULL,
 customer_name VARCHAR(100), segment VARCHAR(40), loyalty_tier VARCHAR(30),
 address VARCHAR(200), city VARCHAR(60), state_name VARCHAR(60),
 effective_from DATE NOT NULL, effective_to DATE NOT NULL,
 current_flag BOOLEAN NOT NULL,
 INDEX idx_customer_business_current(customer_id,current_flag)
);
CREATE TABLE dim_promotion (
 promotion_key INT AUTO_INCREMENT PRIMARY KEY, promotion_code VARCHAR(40) UNIQUE,
 promotion_name VARCHAR(100), promotion_type VARCHAR(40),
 start_date DATE, end_date DATE
);
CREATE TABLE dim_channel (
 channel_key INT AUTO_INCREMENT PRIMARY KEY, channel_name VARCHAR(30) UNIQUE,
 source_system VARCHAR(50)
);
CREATE TABLE dim_delivery_partner (
 delivery_partner_key INT AUTO_INCREMENT PRIMARY KEY,
```

```
 partner_code VARCHAR(40) UNIQUE, partner_name VARCHAR(80)
);
CREATE TABLE fact_sales (
 sales_line_key BIGINT AUTO_INCREMENT PRIMARY KEY,
 order_number VARCHAR(40) NOT NULL, order_line_number INT NOT NULL,
 date_key INT NOT NULL, product_key INT NOT NULL, store_key INT NOT NULL,
 customer_key INT NOT NULL, promotion_key INT NOT NULL, channel_key INT NOT NULL,
 delivery_partner_key INT NOT NULL,
 quantity_sold DECIMAL(12,2) NOT NULL,
 gross_sales_amount DECIMAL(14,2) NOT NULL,
 discount_amount DECIMAL(14,2) NOT NULL,
 net_sales_amount DECIMAL(14,2) NOT NULL,
 unit_selling_price DECIMAL(14,2) NOT NULL,
 FOREIGN KEY(date_key) REFERENCES dim_date(date_key),
 FOREIGN KEY(product_key) REFERENCES dim_product(product_key),
 FOREIGN KEY(store_key) REFERENCES dim_store(store_key),
 FOREIGN KEY(customer_key) REFERENCES dim_customer(customer_key),
 FOREIGN KEY(promotion_key) REFERENCES dim_promotion(promotion_key),
 FOREIGN KEY(channel_key) REFERENCES dim_channel(channel_key),
 FOREIGN KEY(delivery_partner_key) REFERENCES dim_delivery_partner(delivery_partner_key),
 UNIQUE(order_number,order_line_number)
);
CREATE TABLE fact_returns (
 return_line_key BIGINT AUTO_INCREMENT PRIMARY KEY,
 return_number VARCHAR(40) NOT NULL, original_order_number VARCHAR(40),
 date_key INT NOT NULL, product_key INT NOT NULL, store_key INT NOT NULL,
 customer_key INT NOT NULL, quantity_returned DECIMAL(12,2) NOT NULL,
 return_amount DECIMAL(14,2) NOT NULL, return_reason VARCHAR(80),
 FOREIGN KEY(date_key) REFERENCES dim_date(date_key),
 FOREIGN KEY(product_key) REFERENCES dim_product(product_key),
 FOREIGN KEY(store_key) REFERENCES dim_store(store_key),
 FOREIGN KEY(customer_key) REFERENCES dim_customer(customer_key)
);
```

3.6 Why the declared grain matters

The grain is the contract that defines what one fact row means. Every measure must describe that same event. In this design, quantity and line revenue belong to a product line, not to an entire order. A three-line order therefore creates three sales fact rows. The order number is repeated across those lines, so summing a line-level order_count flag would count the same order more than once. COUNT(DISTINCT order_number) or a separate order-header fact is required for order counts.

Dimension attributes should describe the context of the sales line. The product key identifies the version of the product description used by the warehouse; the store key identifies the selling/fulfilment store; the date key identifies the transaction date; and the customer key identifies the historical customer version applicable at the time of purchase. Promotion and channel keys allow comparison of campaign and sales-channel performance.

3.7 Additional modelling rules

Use a special dimension member for Unknown, Not Applicable and (where needed) Late Arriving. These are different business meanings and should not be confused. For example, a store may genuinely be not applicable for a purely digital transaction, while an unknown store indicates that the source should have supplied a store but did not.

Use role-playing dates if the warehouse later needs order date, dispatch date, delivery date and return date. The same DIM_DATE structure can be joined through different foreign keys. This avoids creating unrelated date definitions and ensures consistent calendar attributes.

Promotion attribution must be defined. If one order line can receive multiple simultaneous promotions, a single promotion_key may not be sufficient. In that case, use a bridge table or allocate discount amounts across promotions according to a documented rule. Do not duplicate the sales fact row because doing so would inflate sales totals.

Fact tables should retain source transaction identifiers for traceability and enforce an appropriate uniqueness rule. The sample DDL uses (order_number, order_line_number); in a multi-source environment this should be extended to include source_system because two systems may issue the same order number.

TASK 4 – COMPARE WITH A SNOWFLAKE SCHEMA (4 MARKS)

4.1 Snowflaked dimensions

```
DIM_PRODUCT(product_key, product_code, product_name, brand_key, category_key)
DIM_BRAND(brand_key, brand_name, category_key)
DIM_CATEGORY(category_key, category_name, department_key)
DIM_DEPARTMENT(department_key, department_name)
DIM_STORE(store_key, store_code, store_name, city_key)
DIM_CITY(city_key, city_name, state_key)
DIM_STATE(state_key, state_name, region_key)
DIM_REGION(region_key, region_name)
```

The snowflake moves repeated descriptive attributes into normalized lookup dimensions. Foreign keys replace repeated text values. It introduces extra joins from the fact to the product/store dimension and then through the hierarchy tables.

4.2 Illustrative storage calculation

Assumptions for a transparent estimate: each hierarchy label occupies an average of 20 bytes before row/engine overhead; compare only the repeated text payload, not full physical database size. Product hierarchy labels in a star are repeated for 50,000 products: 50,000 × (20 + 20 + 20) = 3,000,000 bytes. In the snowflake they are stored once: (800 + 60 + 12) × 20 = 17,440 bytes. Estimated product-label saving = 2,982,560 bytes.

Store hierarchy labels in a star: 120 × (20 + 20 + 20) = 7,200 bytes. Snowflake: (70 + 6 + 3) × 20 = 1,580 bytes. Saving = 5,620 bytes. Total illustrative saving = 2,988,180 bytes, approximately 2.85 MiB. Actual savings depend on database encoding, row overhead, indexes, compression and whether labels are already dictionary-compressed. This is a payload estimate, not a benchmark.

4.3 Join-count comparison

Representative query	Star joins (dimension joins)	Snowflake joins	Reason
Net sales by product category and store region	2: fact→product, fact→store	6: fact→product→category and fact→store→city→state→region	Snowflake traverses normalized hierarchies.
Net sales by department and quarter	2: fact→product, fact→date	4: fact→product→category→department and fact→date	More joins to reach department.
Sales by city and product brand	2: fact→store, fact→product	3: fact→store→city and fact→product→brand	One extra hierarchy join.

The star schema is recommended for FreshCart because analysts need interactive dashboards and the dimensions are modest in size (50,000 products and 120 stores). The estimated label-space saving is small relative to fact-table growth and the extra joins. Change to snowflake if dimensions become very large, highly redundant, centrally governed shared hierarchies, or if storage/network transfer becomes a measured bottleneck and query performance remains acceptable.

4.4 Worked query examples and trade-offs

Example 1: category sales by region. In the star schema, the query joins FACT_SALES to DIM_PRODUCT and DIM_STORE, then groups by category_name and region. In the snowflake schema, the product path may continue from product to brand/category and department, while the store path continues from store to city, state and region. The extra joins are the price paid for normalized hierarchy storage.

Example 2: department sales by quarter. The star query joins the fact to date and product. The snowflake query follows product to category and then department. If analysts frequently ask for department-level summaries, the repeated extra joins can affect interactive response time unless the database optimizer and indexes handle them efficiently.

The storage calculation in this assignment counts repeated label payload only. It deliberately excludes row headers, indexes, foreign-key columns, compression and page fill. Therefore it is an estimate to compare the design principle, not a claim about the exact database file size. A fair physical comparison must create both schemas in the same DBMS, load the same members, and compare table plus index sizes.

Business users generally find a star schema easier because the dimension contains familiar descriptive labels in one place. A snowflake may improve consistency when a hierarchy is maintained by a separate master-data team, but it requires users and BI tools to understand more tables and joins. For FreshCart, usability and common query speed outweigh the small estimated saving.

TASK 5 – DESIGN THE ETL PROCESS (5 MARKS)

5.1 ETL flow

```
Source extracts / CDC → Raw staging → Parse & standardize → Validate
        → Lookup / create dimension members → Apply SCD rules
        → Load dimensions → Resolve surrogate keys → Load facts
        → Reconcile counts & amounts → Publish marts / refresh aggregates
```

5.2 Data-quality issues and transformation rules

Source row	Detected issue	Transformation / action
1001	Date 03/11/2026 is ambiguous (DD/MM or MM/DD).	Source locale is India; parse as 03-Nov-2026. Preserve raw value and record parse format.
1002	Customer alias R. Kumar; store case differs; product spacing/case differs; ISO date.	Resolve customer using trusted source customer ID or approved match; normalize store code to CHENNAI-04; map product to canonical Basmati Rice 5kg; parse 2026-11-03.
1003	Negative quantity −3 in a sales extract.	Do not silently convert to positive sale. Route to exception/reversal handling; confirm whether it is a return. If confirmed, load FACT_RETURNS with positive returned quantity 3; otherwise reject pending source correction.
1004	Store is NULL.	Use an Unknown/Unassigned store surrogate key and place row in data-quality queue; do not guess store.
1005	Unit price 9.40 USD differs from INR records.	Convert using the transaction-date approved FX rate and retain original amount/currency and FX rate. If FX rate unavailable, quarantine row. Do not treat 9.40 as INR.
1006	Product is NULL; price uses comma thousands separator.	Parse 1,250 as 1250.00; map product to Unknown Product key and quarantine for product resolution; do not invent product.

5.3 Cleaned / staged extract (illustrative)

Assumptions: source locale is DD/MM/YYYY; the customer alias in row 1002 is only merged if a master-data rule confirms the same customer; the USD conversion row is held until the approved daily exchange rate is available. Therefore the output explicitly distinguishes loadable, quarantined and redirected rows.

Order	Customer	Store	Product	Qty	Unit price / currency	Date	ETL disposition
1001	Rajesh Kumar	CHENNAI-04	Basmati Rice 5kg	2	620.00 INR	2026-11-03	Load after dimension lookup
1002	R. Kumar (match pending)	CHENNAI-04	Basmati Rice 5kg	1	620.00 INR	2026-11-03	Load only after customer

							identity match
1003	Anita Joseph	PUNE-11	Olive Oil 1L	3 returned	780.00 INR	2026-11-04	Redirect to returns fact if source confirms return
1004	Meena S	UNKNOWN STORE	Tea 250g	4	95.50 INR	2026-11-04	Load with unknown store key; exception queue
1005	Anita Joseph	PUNE-11	Olive Oil 1L	3	9.40 USD	2026-11-04	Quarantine until FX conversion is available
1006	Vikram Rao	HYD-02	UNKNOWN PRODUCT	1	1250.00 INR	2026-11-05	Load with unknown product key only if policy permits; exception queue

5.4 Surrogate keys, SCD and load sequence

- For each dimension row, ETL looks up its source business key and current record. It assigns the dimension's surrogate key to the fact row.
- Unknown or late-arriving members use a reserved surrogate key (for example 0 = Unknown). When the real member arrives, update the placeholder or restate affected facts according to the warehouse's late-arriving-dimension policy. Keep an exception log.
- Customer address and loyalty tier use SCD Type 2 because management asks for the address and tier that applied when an order occurred. On change, expire the old row (effective_to, current_flag = false) and insert a new row with a new customer_key. Facts resolve the version effective on the order date.
- Initial load: extract historical source data, clean it, load dimensions first (including Unknown members), then facts, then aggregates and BI views.
- Incremental load: use CDC or updated_at/watermark, stage only changed/new records, deduplicate by source transaction key, update dimensions/SCD, resolve keys, upsert facts, reconcile and refresh aggregates.
- Audit: record batch_id, source row count, accepted count, rejected count, inserted/updated dimension counts, fact count, amount totals, start/end timestamps and rejection reasons. Require source_count = accepted + rejected.

5.5 Detailed transformation and validation sequence

Step 1 – land raw data: retain the original values exactly as received, including mixed date formats, nulls, spelling differences and currency symbols. Store the source file name, source system, extraction time and batch ID. This raw layer is immutable so that later corrections can be reproduced.

Step 2 – standardize: trim leading/trailing spaces, normalize case for codes, map known store aliases to canonical store codes, standardize product descriptions through a product master crosswalk, parse dates using source-specific locale rules and convert numeric strings into decimal values. Never infer a missing business key from a person's name alone.

Step 3 – validate: check mandatory fields, positive sale quantities, non-negative prices, valid store/product references, supported currencies and date ranges. A negative quantity is not automatically a valid return: the source transaction type must confirm that it is a return or reversal. Invalid or ambiguous records enter a quarantine table.

Step 4 – resolve dimensions: lookup the customer, product, store, date, promotion, channel and delivery-partner surrogate keys. If a dimension member is not yet available, assign the agreed Unknown/Late Arriving key and retain the natural key in the exception queue. Retry resolution in a later batch.

Step 5 – load facts and reconcile: insert only records that pass validation and key resolution. Use idempotent batch logic so rerunning a batch does not duplicate facts. Compare source and target row counts, quantity totals and monetary totals by source system and batch.

5.6 SCD Type 2 example

Suppose customer C102 has address '12 Lake Road' and loyalty tier 'Silver' until 2026-11-10. On 2026-11-11 the customer changes address to '8 Park Street' and tier to 'Gold'. The old dimension row is closed with effective_to = 2026-11-10 and current_flag = false. A new row is inserted with a new customer_key, effective_from = 2026-11-11, effective_to = 9999-12-31 and current_flag = true.

A sale dated 2026-11-08 resolves to the old customer_key, while a sale dated 2026-11-12 resolves to the new customer_key. Thus historical reports can group the old order under the Silver tier and old address even after the customer's profile changes. The ETL lookup must use the transaction timestamp between effective_from and effective_to, not simply the current_flag.

5.7 Rejection and audit design

Keep a rejected_records table with batch_id, source_system, source_record_id, raw_payload, rejection_code, rejection_description and rejected_timestamp. Typical reason codes include INVALID_DATE, UNKNOWN_CURRENCY, NEGATIVE_SALE_QTY, MISSING_PRODUCT and UNMATCHED_CUSTOMER. Data stewards can correct master data or request a source-system fix, after which the row can be replayed.

At the end of each batch, publish a reconciliation report containing source rows, duplicate rows, accepted rows, rejected rows, inserted facts, updated dimensions, gross sales, discounts, returns and net sales. The basic count equation is source rows = accepted rows + rejected rows + explicitly identified duplicates. Financial totals should be reconciled separately because rejected rows may carry amounts that must be explained.

TASK 6 – MODEL THE DATA CUBE AND APPLY OLAP OPERATIONS (5 MARKS)

6.1 Cuboid lattice calculation

With the stated hierarchies, each dimension can be represented at one of its listed levels or at ALL. Time has 4 levels + ALL = 5 choices; Product has 3 + ALL = 4; Store has 3 + ALL = 4; Customer Segment has 1 + ALL = 2. Total cuboids = 5 × 4 × 4 × 2 = 160.

Without hierarchy levels, each of the four dimensions is either included at its base level or aggregated to ALL: $2^4 = 16$ cuboids. Hierarchies increase the lattice from 16 to 160.

Base cuboid: day × product × store × customer segment, at the most detailed supported level. Apex cuboid: ALL dimensions, containing the overall aggregate (for example total net sales).

6.2 OLAP operations with business questions and SQL

Assume fact_sales is joined to the dimensions described in Task 3 and that net_sales_amount is stored at sales-line grain. The SQL below illustrates the equivalent relational operation.

Roll-up

Question: What are net sales by quarter instead of day? Cuboid: Quarter × Department × Region × Segment.

```
SELECT d.year_num, d.quarter_num, p.department_name, s.region,
       SUM(f.net_sales_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON f.date_key=d.date_key
JOIN dim_product p ON f.product_key=p.product_key
JOIN dim_store s ON f.store_key=s.store_key
GROUP BY d.year_num, d.quarter_num, p.department_name, s.region;
```

Drill-down

Question: For a selected quarter, which months contributed to the quarter's sales? Cuboid: Month × Department × Region, filtered to one year/quarter.

```
SELECT d.year_num, d.month_num, p.department_name, SUM(f.net_sales_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON f.date_key=d.date_key
JOIN dim_product p ON f.product_key=p.product_key
WHERE d.year_num=2026 AND d.quarter_num=4
GROUP BY d.year_num, d.month_num, p.department_name;
```

Slice

Question: What is sales performance for online channel only? Cuboid: Day × Product × Store × Segment with Channel fixed to Online.

```
SELECT d.year_num, d.month_num, SUM(f.net_sales_amount) AS online_sales
FROM fact_sales f JOIN dim_date d ON f.date_key=d.date_key
JOIN dim_channel c ON f.channel_key=c.channel_key
WHERE c.channel_name='Online'
GROUP BY d.year_num, d.month_num;
```

Dice

Question: Compare selected departments in two regions for Q3 and Q4. Cuboid: Quarter × Department × Region, restricted to selected values.

```
SELECT d.quarter_num, p.department_name, s.region, SUM(f.net_sales_amount) AS net_sales
FROM fact_sales f JOIN dim_date d ON f.date_key=d.date_key
JOIN dim_product p ON f.product_key=p.product_key
JOIN dim_store s ON f.store_key=s.store_key
WHERE d.year_num=2026 AND d.quarter_num IN (3,4)
AND p.department_name IN ('Grocery','Personal Care')
AND s.region IN ('North','South')
GROUP BY d.quarter_num, p.department_name, s.region;
```

Pivot

Question: Present department sales with regions as columns for a management comparison. Cuboid: Department × Region, displayed in a rotated layout. Conditional aggregation is a portable SQL pivot equivalent.

```
SELECT p.department_name,
 SUM(CASE WHEN s.region='North' THEN f.net_sales_amount ELSE 0 END) AS north_sales,
 SUM(CASE WHEN s.region='South' THEN f.net_sales_amount ELSE 0 END) AS south_sales,
 SUM(CASE WHEN s.region='West' THEN f.net_sales_amount ELSE 0 END) AS west_sales
FROM fact_sales f JOIN dim_product p ON f.product_key=p.product_key
JOIN dim_store s ON f.store_key=s.store_key
GROUP BY p.department_name;
```

6.3 Partial cuboid materialisation

If storage permits only a small fraction of 160 cuboids, materialise the cuboids most frequently used by dashboards: (1) Month × Department × Region, (2) Day × Store, (3) Quarter × Category × Region, and (4) Month × Channel × Region. These support the stated management questions and reduce repeated scans. Keep the base fact table for drill-down and less common combinations. Refresh summaries after each hourly ETL batch. Track summary-table storage and refresh time as part of total warehouse cost.

6.4 Understanding the cuboid lattice

A cuboid is one particular aggregation level across the dimensions. For example, Day × Product × Store × Segment is the base cuboid, while Quarter × Department × State × Segment is a less detailed cuboid. The ALL value means that dimension has been aggregated away. The apex has ALL for every dimension and stores the grand total.

The 160-cuboid calculation assumes one selectable level from each stated hierarchy plus ALL. It counts combinations of levels, not the number of possible data rows inside each cuboid. A hierarchy-aware lattice therefore has more possible aggregation views than a simple four-dimensional binary include/exclude lattice.

6.5 Choosing materialized cuboids

Materialization should be driven by workload frequency, query response-time target, refresh cost and storage. A useful first choice is a monthly department-by-region cuboid because it supports strategic trend dashboards and is much smaller than the base fact. A daily store cuboid supports store managers, while a quarter/category/region cuboid supports quarter-over-quarter category analysis.

Do not materialize every possible cuboid. Some cuboids overlap and create redundant storage. The base fact remains the source of truth for drill-down and unusual filters. After observing actual dashboard usage, retain aggregates that are frequently queried and expensive to compute; remove rarely used summaries. Each aggregate needs a documented grain and refresh timestamp.

6.6 Correctness considerations

Returns should not be subtracted twice. Either store net sales at line grain after allocating returns, or calculate gross sales less discounts and subtract the returns fact in a reporting view. The assignment design keeps sales-line measures and a separate returns fact, so a net-sales report must join or aggregate returns at a compatible grain before subtraction.

Pivot changes the presentation of a cuboid, not its underlying measure. A pivoted report should reconcile to the same grand total as the non-pivoted grouped result. Slice and dice filters must be applied before aggregation, and drill-down should expose more detailed levels without changing the total for the parent group.

TASK 7 – DEVELOP AN OLAP APPLICATION (5 MARKS)

A runnable starter application is supplied in the companion ZIP. It generates synthetic FreshCart sales-line data at selectable scales (1,000; 10,000; 100,000; and 1,000,000 rows) and provides interactive Roll-up, Drill-down, Slice, Dice and Pivot operations. Users choose dimensions, grouping level and filters. The app uses Pandas for aggregation and Plotly for charts.

7.1 Run instructions

```
python -m venv .venv
# Windows: .venv\\Scripts\\activate
pip install -r requirements.txt
python generate_data.py --rows 10000
streamlit run app.py
```

The generator creates synthetic data, not actual FreshCart transactions. The seed is fixed for repeatability. Run the generator at each required scale and save the output row count and file size. A one-million-row run may require several minutes and sufficient RAM.

7.2 Validation / reconciliation

The application displays the row count and total net sales from the generated source file. For each OLAP view, reconcile the sum of grouped net_sales to the independent Pandas sum of the ungrouped source. For filters, compare the displayed total with a separately filtered source total. A successful reconciliation should have a difference of 0.00 (subject to consistent decimal rounding). Save screenshots showing each operation, filter selections, output table and reconciliation result.

7.3 Application test cases and screenshots

Test case 1 – Roll-up: select Time and Quarter. Confirm the app groups daily records into quarter totals. Test case 2 – Drill-down: select Time and Month or Day and confirm that a selected parent period can be inspected at finer granularity. Test case 3 – Slice: select one channel, such as Online, and verify that only online rows contribute.

Test case 4 – Dice: apply at least two filters simultaneously, such as Region = South/West and Department = Grocery/Personal Care. Test case 5 – Pivot: place Region values into separate columns and verify that the sum across pivot columns equals the filtered source total. Capture one screenshot per operation with the selected inputs visible.

Also capture a screenshot of the dataset generation terminal showing the row count, the app's total fact rows, and the reconciliation result. Use the same generated CSV for all screenshots so the figures remain consistent.

7.4 Data scale and repeatability

Run the generator at 1,000, 10,000, 100,000 and 1,000,000 rows. Keep the random seed fixed to make results reproducible. Record the file size, generation time, application load time and aggregation response time. The synthetic dataset is suitable for demonstrating the interface and aggregation behavior; it is not a substitute for confidential production data or evidence of real FreshCart sales.

The one-million-row test may be more efficient if the app reads only required columns or uses database-side aggregation. If the machine runs out of memory, document the hardware and use chunked processing or a database backend. A good assignment report explains the limitation rather than hiding it.

TASK 8 – EVALUATE QUERY PROCESSING PERFORMANCE (4 MARKS)

Use the companion benchmark script and run it at each dataset size. Compare: (i) normalized OLTP-style joins, (ii) star-schema joins, and (iii) star schema with indexes/aggregate summary. Record actual results from the same computer, database version, cache state and data volume. Do not copy invented timings.

8.1 Representative queries

Query	Analytical purpose	Expected performance factors
Q1	Net sales by quarter, category and region.	Fact rows scanned, date/product/store joins, grouping cardinality; date and foreign-key indexes or partitioning can help.
Q2	Top 10 products by net sales for a selected region and year.	Selective date/region filters, product join, sort/top-N; composite indexes and pre-aggregates may reduce work.
Q3	Return quantity and return rate by customer segment and product category.	Joins across sales/returns and dimensions; correct grain and distinct denominators are essential.

8.2 Results table to fill after execution

Rows	Query	Schema / optimization	Time (ms)	Rows read/scanned	Joins	Plan / notes
1,000	Q1	Normalized / Star / Star+index-summary	Record run	Record plan	Record	Attach EXPLAIN output
10,000	Q1	Normalized / Star / Star+index-summary	Record run	Record plan	Record	Attach EXPLAIN output
100,000	Q1	Normalized / Star / Star+index-summary	Record run	Record plan	Record	Attach EXPLAIN output
1,000,000	Q1	Normalized / Star / Star+index-summary	Record run	Record plan	Record	Attach EXPLAIN output
Repeat for Q2 and Q3 at each scale.						

Use EXPLAIN (or EXPLAIN ANALYZE where supported) to capture access type, estimated/actual rows, chosen indexes and join order. Also record index/summary storage and the time required to build or refresh them. A summary table speeds repeated aggregates but costs disk space, refresh CPU and additional ETL complexity; stale summaries can produce incorrect dashboards if refresh is delayed.

8.3 Experimental method

Use identical logical data and identical filter values for all three designs. Warm up each query once, then run it at least five times and report the median time; also note the cold-cache result separately if possible. Run tests on the same machine, with the same DBMS version, memory settings and indexes. Record whether query cache is enabled.

For each query, save EXPLAIN or EXPLAIN ANALYZE output. Record estimated and actual rows where available, table access method, indexes selected, join order and temporary sort/group operations. Execution time alone cannot explain why a query is fast or slow; rows scanned and plan shape provide supporting evidence.

Measure the cost of optimization too. Record index size, summary-table size, summary build time and incremental refresh time. A summary table can make a dashboard query much faster but may add a significant hourly load cost. The correct design minimizes total workload cost while meeting freshness requirements.

8.4 Interpreting expected patterns

The normalized OLTP-style schema is expected to require more joins because customer, product, category, store and transaction details are split across operational tables. The star schema usually simplifies analytical joins by presenting denormalized dimensions around the fact. Indexes can improve selective filters, but a full scan may still be optimal for a query that reads a large fraction of the fact table.

An aggregate table can reduce millions of fact rows to a smaller set of pre-grouped rows. Its benefit is greatest for repeated queries that match its grain. It may not help an unusual query that needs a detail attribute not included in the summary. If summaries are not refreshed after new facts arrive, dashboards may be fast but stale.

TASK 9 – BUILD A KNIME REPORTING WORKFLOW (3 MARKS)

9.1 Proposed node sequence

KNIME node	Purpose	Warehouse concept
CSV Reader / DB Connector + DB Table Selector	Read fact and dimension data from CSV or MySQL.	Source extraction / data access
String Manipulation	Trim text, normalize case and standardize codes.	ETL cleansing
Missing Value	Handle nulls using approved Unknown values or mark rows for review.	Data-quality handling
Rule Engine / Column Expressions	Create weekend flag, channel labels or derived metrics.	ETL transformation
Joiner	Join fact data with product/store/date dimensions when reading separate files.	Dimension lookup / relational join
Row Filter	Filter year, region, channel or department.	OLAP slice/dice
GroupBy	Sum net sales and quantity; count orders appropriately.	OLAP aggregation / roll-up
Pivoting	Convert region values into columns.	OLAP pivot
Bar Chart / Line Plot	Visualize category trends or month-on-month sales.	BI presentation
Excel Writer / CSV Writer	Export a management report.	Report delivery

Workflow: Reader → String Manipulation → Missing Value → Rule Engine → Joiner → Row Filter → GroupBy → Pivoting → chart → Excel Writer. Add a separate rejected-row branch from validation nodes to a CSV Writer so data-quality exceptions remain auditable.

At 1,000,000+ rows, in-memory processing can consume significant RAM and become slower. Prefer database-side filtering and aggregation, DB GroupBy/SQL queries, columnar formats, and incremental processing rather than loading every row into memory.

Evidence to attach: screenshot of the complete KNIME node canvas, screenshot of the configured GroupBy/Pivoting nodes, and the exported report file. The companion package includes a synthetic CSV to use for the workflow.

9.2 Suggested workflow branches

Create a clean-data branch and a rejected-data branch. After the Reader node, use rule-based checks for missing product codes, invalid dates and non-positive sale quantities. Valid rows continue to the transformation and aggregation path; invalid rows are sent to a separate output with a rejection reason. This mirrors the quarantine design from Task 5.

On the valid branch, derive year, month, quarter and weekend/weekday fields. Join the fact extract to product and store reference data if the CSV contains surrogate keys rather than labels. Use Row Filter to select the required time period or region, GroupBy to sum net sales and quantity, and Pivoting to compare regions as columns. Finish with a chart and Excel Writer.

To make the workflow auditable, annotate nodes with their purpose, name output tables clearly and save the workflow together with the input CSV. Record the KNIME version and any extensions used. The report should show the generated timestamp and selected filters.

9.3 Validation of the KNIME report

Compare the GroupBy grand total with the SQL or Python source total for the same filter conditions. Confirm that the Pivoting node changes only the layout and not the grand total. Test missing values and an empty filter result so the workflow handles edge cases cleanly.

At one million rows, KNIME's in-memory nodes may require more heap memory and can become slower. Database pushdown, DB GroupBy, column pruning and filtering early in the workflow reduce data transfer. If a workflow uses CSV, a columnar format or database connection may be more suitable at larger scale.

TASK 10 – RECOMMEND AND DEFEND THE FINAL DESIGN (3 MARKS)

Recommend an enterprise warehouse with a dimensional star schema and dependent sales/returns marts. Use conformed Date, Product, Store and Customer dimensions, plus Promotion, Channel and Delivery Partner dimensions. Use SCD Type 2 for customer address and loyalty tier so historical facts retain the attributes effective at transaction time. Load data incrementally and refresh aggregates hourly.

The star schema is preferred to the snowflake because the additional storage saved by normalizing the stated hierarchy labels is modest under the stated assumptions, while the snowflake adds joins to common management queries. Performance claims must be supported by the benchmark results collected in Task 8. Indexes and selected aggregate tables are recommended only when EXPLAIN plans and measured query times show a benefit.

At 1,000,000+ rows, a fact table with appropriate indexes, partitioning by date when supported, incremental loads and selective aggregates remains practical. Monitor ETL duration, dashboard freshness, failed/rejected rows, query latency, fact growth, index size and aggregate refresh time.

10.1 Decision matrix

Enterprise warehouse versus virtual warehouse: select the enterprise warehouse because it integrates multiple source systems, stores history and isolates analysis from billing. Independent data marts are rejected because separate definitions of product, customer and net sales would create inconsistent results. Dependent marts preserve shared conformed dimensions.

Star versus snowflake: select star for the initial implementation because the product and store hierarchies are relatively small and common reports require fast, understandable joins. The snowflake remains a valid alternative if measured dimension storage or master-data governance becomes a stronger concern.

Full precomputation versus selective summaries: select only workload-driven aggregates. The fact tables preserve detail and support new questions; summary tables accelerate repeated dashboards. This balances query latency against storage and hourly refresh cost.

10.2 Risks and controls

Key risks include inconsistent source identifiers, ambiguous dates, currency conversion errors, late-arriving dimensions, incorrect return allocation and stale aggregates. Controls include source crosswalks, locale-aware parsing, approved FX rates, SCD effective dates, reconciliation, exception queues and batch-level freshness metadata.

The design should be reviewed using evidence from Tasks 4 and 8. The estimated snowflake storage saving is an analytical estimate; the benchmark is the empirical evidence for query behavior. If measurements contradict the expectation, revise the recommendation rather than forcing the result to fit the original choice.

TASK 11 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT (8 MARKS)

11.1 A. Impact analysis

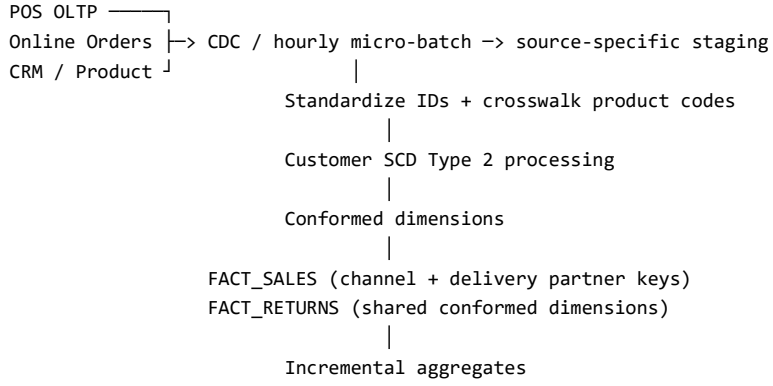
Changed requirement	Affected components	Impact
New online source: ~20,000 orders/day	Source ingestion, staging, channel dimension, fact loading	Add source adapter and deduplication by source system + order ID.
Different product-code format	Product dimension and ETL mapping	Add source-system product cross-reference; map multiple source codes to one enterprise product key.
New delivery partner attribute	Fact/dimension structure	Add DIM_DELIVERY_PARTNER and delivery_partner_key to FACT_SALES; use Unknown/Not Applicable members.
Address and loyalty tier change frequently	DIM_CUSTOMER and ETL	Use SCD Type 2; resolve customer version effective on order timestamp.
Hourly dashboard refresh	Scheduling, CDC/micro-batch, aggregates and BI cache	Run incremental micro-batches hourly; publish only after reconciliation.

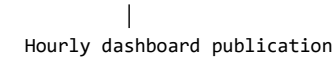
The core sales-line grain, date/product/store/customer dimensional approach, returns fact and OLAP operations remain valid. Structure changes are limited to channel/delivery partner and customer-history attributes. Load-process changes include multi-source ingestion, source-code crosswalks, CDC and hourly micro-batches. Performance changes include increased ingestion rate and more frequent aggregate refresh.

11.2 B. Evaluate the original design

At 20,000 online orders per day, the average arrival rate is approximately 833 orders per hour (20,000 ÷ 24), before peak-hour variation. If the average order contains 3 lines, the added load is approximately 60,000 fact rows/day or 2,500 fact rows/hour. These are planning assumptions; measure the actual average lines per order and peak rate. An overnight full reload would not meet the hourly dashboard requirement. The original star schema can handle the added channel if its ingestion, keys and refresh schedule are extended; the original single-source ETL and Type 1/current-only customer dimension would fail historical address/tier reporting.

11.3 C. Revised design





Add source_system and source_order_id to staging/audit records and enforce a unique business key such as (source_system, order_id, order_line_id). Add a product cross-reference table (source_system, source_product_code, product_key, valid_from, valid_to). Add delivery_partner_key to sales facts. Customer Type 2 rows store effective_from/effective_to and current_flag; facts use the version valid at the order timestamp.

11.4 D. Evidence-based defence and monitoring

The arithmetic above establishes a baseline expected hourly load, not a measured throughput result. Validate by replaying generated batches representing the observed daily order count and line count. Record extraction duration, staging rows, accepted/rejected rows, dimension lookup time, fact insert time, aggregate refresh time, dashboard query latency and total end-to-end freshness. The design is acceptable only if the full pipeline completes within the one-hour refresh window with operational headroom.

Monitor: (1) batch completion and data freshness; (2) source-to-warehouse row and amount reconciliation; (3) duplicate business keys; (4) unknown product/store/customer key rates; (5) SCD Type 2 version overlaps/gaps; (6) hourly query latency; (7) fact and index growth; (8) failed/rejected rows; and (9) aggregate refresh duration. Alert if a batch misses its SLA or reconciliation differs.

11.5 Handling source product codes and identity resolution

Create a PRODUCT_SOURCE_XREF table with source_system, source_product_code, product_key, valid_from and valid_to. A code such as WEB-000123 can map to the same enterprise product_key as a POS code such as 00123. This keeps the fact table conformed while allowing each source to retain its own identifier. Unknown codes are quarantined for master-data review.

Customer matching should use a stable customer identifier supplied by the CRM or identity service. Names and addresses are not reliable unique identifiers. When an online order arrives before the CRM profile, load it with a late-arriving customer member and resolve it when the trusted identity becomes available.

11.6 Hourly micro-batch control

An hourly batch should have a clear sequence: capture source watermark; extract inserts and updates; land immutable raw data; deduplicate by source-system transaction key; standardize and validate; process SCD changes; load dimensions; resolve fact keys; insert facts; update aggregate tables; reconcile; then publish the new dashboard watermark. This sequence prevents a dashboard from exposing partially loaded dimensions and facts.

Use a retryable batch ID and idempotent upsert logic. If a failure occurs after facts are loaded but before aggregates are refreshed, the next attempt should refresh the aggregates without inserting the same facts again. Keep the previous published batch visible until the new batch passes validation.

11.7 Capacity calculation and operational headroom

The assignment states approximately 20,000 online orders per day. The average is 20,000 ÷ 24 ≈ 833 orders/hour. If average line count is 3, this is approximately 60,000 sales lines/day and 2,500 lines/hour. Peak rates can be several times the average, so capacity planning must use measured peak-hour traffic, not only the daily average.

To test the hourly SLA, replay batches at the expected average and a chosen peak multiplier, then measure end-to-end processing time. A practical target is for the batch to complete comfortably before the next hourly run, leaving time for retries and dashboard queries. The exact safety margin should be agreed with FreshCart rather than assumed.

11.8 Deployment monitoring and acceptance criteria

Acceptance criteria should include: no duplicate source transaction keys; source-to-warehouse counts reconcile; sales and returns reconcile by source and day; SCD effective intervals do not overlap; unknown-key rates remain below an agreed threshold; and the dashboard's published watermark is no more than one refresh interval behind the source.

Monitor batch duration, queue/backlog size, source lag, fact insert throughput, dimension lookup failures, rejected records, aggregate refresh duration, query p95 latency, database storage and index growth. Review these metrics after launch and tune indexes, partitions, batch sizes or aggregate selection based on measured bottlenecks.

CONCLUSION

FreshCart should separate transaction processing from decision support by adopting an enterprise warehouse with a star schema, conformed dimensions and governed ETL. The design supports the required retail questions, historical customer analysis, sales and returns reporting, and the five OLAP operations. A snowflake schema offers some dimension-label storage reduction but adds joins; under the stated scale, the star schema is the more usable starting point. The online channel can be supported by extending the original design with source mapping, a channel and delivery-partner dimension, SCD Type 2 customer history and hourly incremental loads. Final performance claims must be based on the measurements collected using the supplied scripts.